Ridge Regression

This is basically a polynomial regression except it helps with overfitting following the concept, low bias low variance

The ordinary least squares (OLS) estimator of β is given by:

$$\hat{\beta}_{\text{OLS}} = (X'X)^{-1}X'y$$

In the presence of multicollinearity, $X'X$ is nearly singular, leading to unstable estimates. ridge regression addresses this issue by adding a penalty term kI, where k is the ridge parameter and I is the identity matrix. The ridge regression estimator is:

$$\hat{\beta}_k = (X'X + kI)^{-1}X'y$$

This modification stabilizes the estimates by shrinking the coefficients, improving generalization and mitigating multicollinearity effects.

With SKLearn

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import r2_score, mean_squared_error

# Load the diabetes dataset
data = load_diabetes()
print(data.DESCR)

# Extract features and target variable
X = data.data
```

```
y = data.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2, random_state=45)

# Linear Regression Model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Display coefficients and intercept for Linear Regression
print("Linear Regression Coefficients:", linear_model.coef_)
print("Linear Regression Intercept:", linear_model.intercept
_)

# Predictions using Linear Regression
y_pred_linear = linear_model.predict(X_test)

# Metrics for Linear Regression
print("Linear Regression R2 Score:", r2_score(y_test, y_pred_
linear))
print("Linear Regression RMSE:", np.sqrt(mean_squared_error(y
_test, y_pred_linear)))

# Ridge Regression Model
ridge_model = Ridge(alpha=100000)
ridge_model.fit(X_train, y_train)

# Display coefficients and intercept for Ridge Regression
print("Ridge Regression Coefficients:", ridge_model.coef_)
print("Ridge Regression Intercept:", ridge_model.intercept_)

# Predictions using Ridge Regression
y_pred_ridge = ridge_model.predict(X_test)

# Metrics for Ridge Regression
```

```
print("Ridge Regression R2 Score:", r2_score(y_test, y_pred_r
idge))
print("Ridge Regression RMSE:", np.sqrt(mean_squared_error(y_
test, y_pred_ridge)))

# Synthetic Dataset for Visualization
m = 100
x1 = 5 * np.random.rand(m, 1) - 2
x2 = 0.7 * x1 ** 2 - 2 * x1 + 3 + np.random.randn(m, 1)

# Scatter plot of synthetic data
plt.scatter(x1, x2, color='blue', alpha=0.5, label='Synthetic
Data')
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Synthetic Data Visualization')
plt.legend()
plt.show()
```

From scratch:

only change is in the equation of m from linear regression

For 2D

$$L = \sum_{i=1}^{n} (y_i - m x_i - \bar{y} + m\bar{x})^2 + \lambda m^2$$

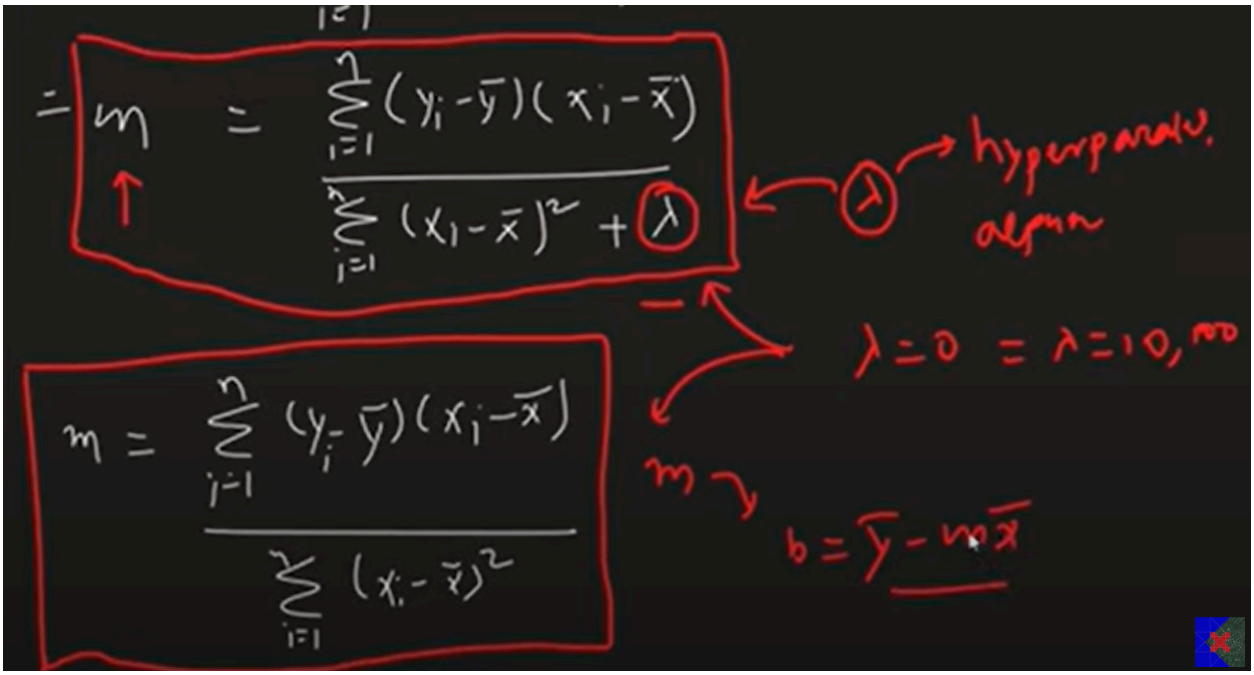
$$\frac{\partial L}{\partial m} = 2 \sum_{i=1}^{n} (y_i - m x_i - \bar{y} - m\bar{x})(-x_i + \bar{x}) + 2\lambda m = 0$$

$$= \lambda m - \sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x}) - m (x_i - \bar{x})^2 = 0$$

$$= \lambda m - \sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x}) + m \sum_{i=1}^{n} (x_i - \bar{x})^2 = 0$$

$$= \lambda m + m \sum_{i=1}^{n} (x_i - \bar{x})^2 = \sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x})$$

$$= m = \frac{\sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^{n} (x_i - \bar{x})^2 + \lambda}$$




```python
# Import necessary libraries
from sklearn.datasets import make_regression
import matplotlib.pyplot as plt
import numpy as np

# Generate synthetic regression data
X, y = make_regression(
    n_samples=100, n_features=1, n_informative=1,
    n_targets=1, noise=20, random_state=13
)

# Visualize the data
plt.scatter(X, y)
plt.title("Scatter Plot of the Data")
plt.xlabel("Feature")
plt.ylabel("Target")
plt.show()

# Import LinearRegression and Ridge from sklearn
```

```
from sklearn.linear_model import LinearRegression, Ridge

# Fit a simple Linear Regression model
lr = LinearRegression()
lr.fit(X, y)

# Display the coefficients of the Linear Regression model
print("Linear Regression Coefficients:")
print("Slope (m):", lr.coef_[0])
print("Intercept (b):", lr.intercept_)

# Fit Ridge Regression models with different alpha values
rr = Ridge(alpha=10)
rr.fit(X, y)
print("\nRidge Regression (alpha=10) Coefficients:")
print("Slope (m):", rr.coef_[0])
print("Intercept (b):", rr.intercept_)

rr1 = Ridge(alpha=100)
rr1.fit(X, y)
print("\nRidge Regression (alpha=100) Coefficients:")
print("Slope (m):", rr1.coef_[0])
print("Intercept (b):", rr1.intercept_)

# Plot the results
plt.plot(X, y, 'b.', label="Data")
plt.plot(X, lr.predict(X), color='red', label='Linear Regress
ion (alpha=0)')
plt.plot(X, rr.predict(X), color='green', label='Ridge Regres
sion (alpha=10)')
plt.plot(X, rr1.predict(X), color='orange', label='Ridge Regr
ession (alpha=100)')
plt.title("Regression Models Comparison")
plt.xlabel("Feature")
plt.ylabel("Target")
plt.legend()
```

```
plt.show()

# Custom implementation of Linear Regression with Ridge regul
arization
def linear_regression(X, y, alpha=1):
    """
    Computes the slope (m) and intercept (b) of a linear regr
ession line
    with optional L2 regularization (Ridge Regression).
    """
    x_mean = X.mean()
    y_mean = y.mean()

    num = 0
    den = 0

    # Compute the numerator and denominator for the slope (m)
    for i in range(X.shape[0]):
        num += (y[i] - y_mean) * (X[i] - x_mean)
        den += (X[i] - x_mean) * (X[i] - x_mean)

    # Add L2 regularization term (alpha)
    m = num / (den + alpha)
    b = y_mean - m * x_mean
    return m, b

# Calculate slope and intercept for different alpha values
m, b = linear_regression(X, y, alpha=0)
print("\nCustom Linear Regression Results (alpha=0):")
print("Slope (m):", m)
print("Intercept (b):", b)

m1, b1 = linear_regression(X, y, alpha=10)
print("\nCustom Ridge Regression Results (alpha=10):")
print("Slope (m):", m1)
print("Intercept (b):", b1)
```

```
# Plot the custom linear regression lines
x_ = np.linspace(X.min(), X.max(), 100)  # Create a range for
the line
plt.plot(X, y, 'b.', label="Data")
plt.plot(x_, m * x_ + b, label='Custom Linear Regression (alp
ha=0)', color='red')
plt.plot(x_, m1 * x_ + b1, label='Custom Ridge Regression (al
pha=10)', color='green')
plt.title("Custom Linear and Ridge Regression Lines")
plt.xlabel("Feature")
plt.ylabel("Target")
plt.legend()
plt.show()
```

For n dim or multilinear ridge regession

as we can see the normal LR multilinear formula just added lemda * (w)^2

Normal LR → Ridge

$$L = \boxed{(XW-Y)^T(XW-Y) + \lambda \|W\|^2}$$

$\lambda w_0^2 + \lambda w_1^2 + \lambda w_2^2 + \dots + \lambda w_n^2$

$$[w_0\ w_1\ w_2 \dots w_n] \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix} \quad \underline{W^T W} \rightarrow \lambda(w_0^2 + w_1^2 + w_2^2 \dots w_n^2)$$

$$\frac{dL}{dW}$$

$$L = (XW-Y)^T(XW-Y) + \lambda W^T \underline{W}$$

$$(a-b)^T = a^T - b^T$$

$$\frac{dL}{dW}$$

$$(ab)^T = b^T a^T$$

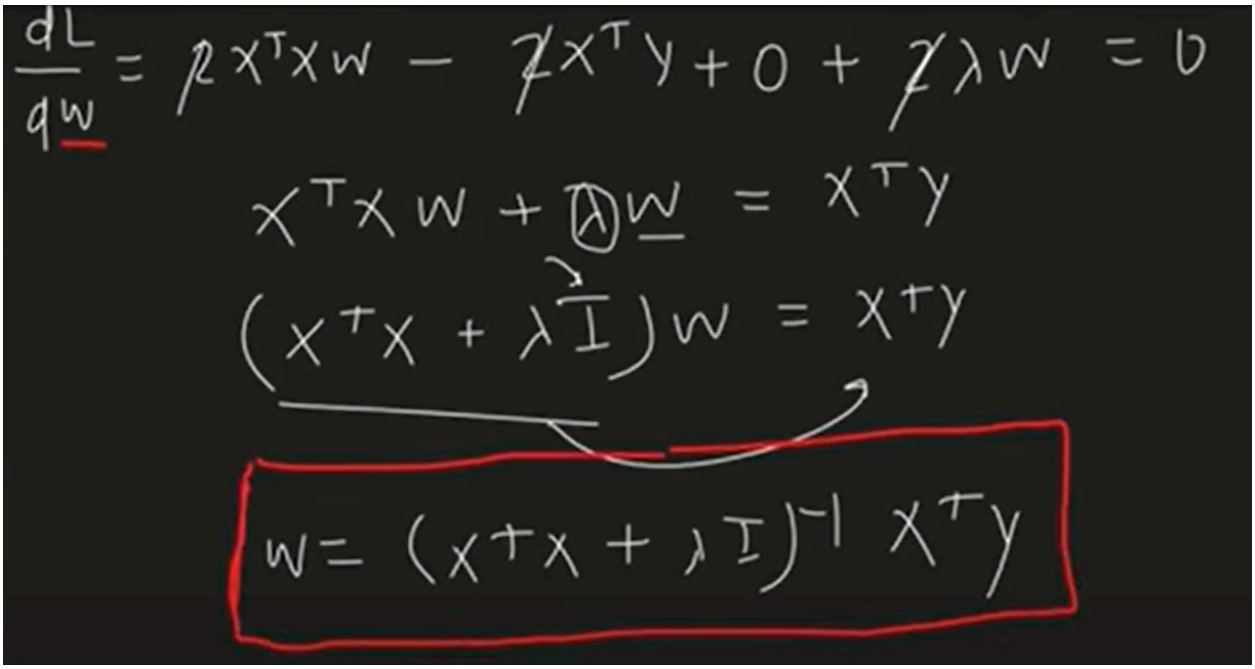
$$L = \underline{(XW-Y)^T(XW-Y)} + \lambda W^T \underline{W}$$

$$L = [(XW)^T - (Y)^T](XW-Y) + \lambda W^T W$$

$$= (W^T X^T - Y^T)(XW-Y) + \lambda W^T W$$

$$= W^T X^T X W - \underbrace{W^T X^T Y - Y^T X W}_{} + Y^T Y + \lambda W^T W$$

$$= W^T X^T X W - 2 W^T X^T Y + Y^T Y + \lambda W^T W$$

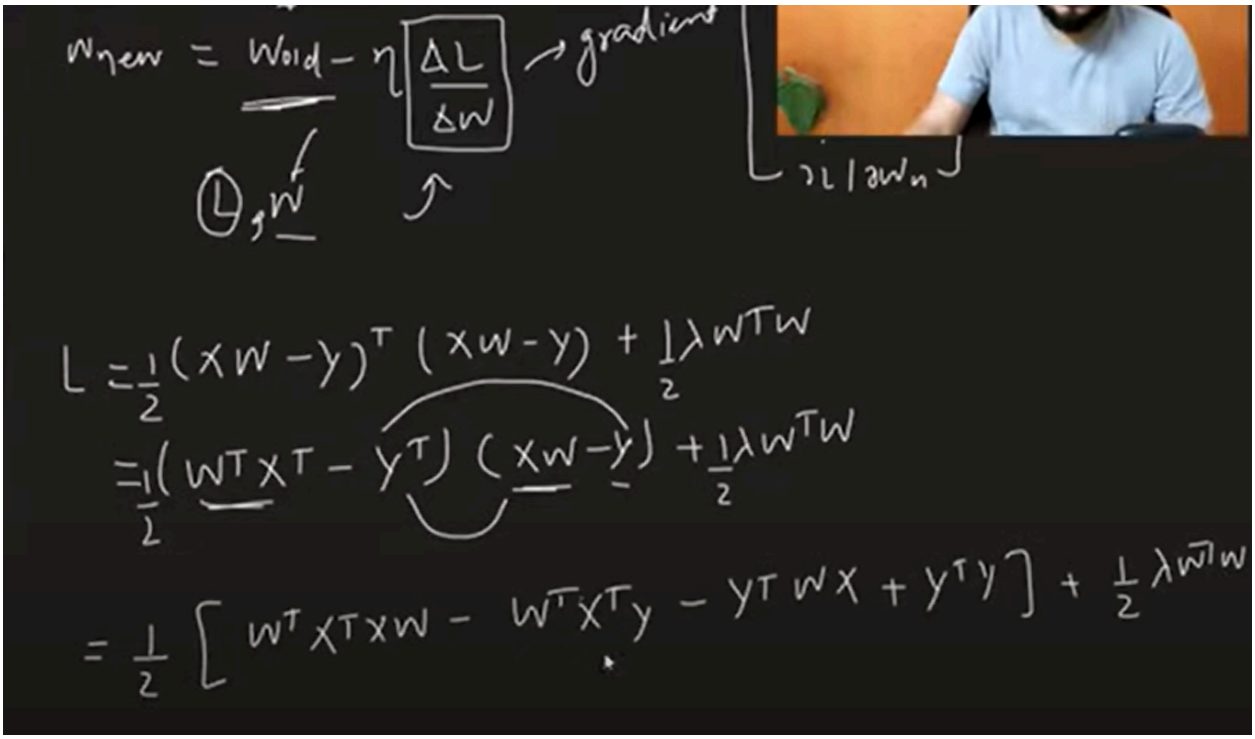


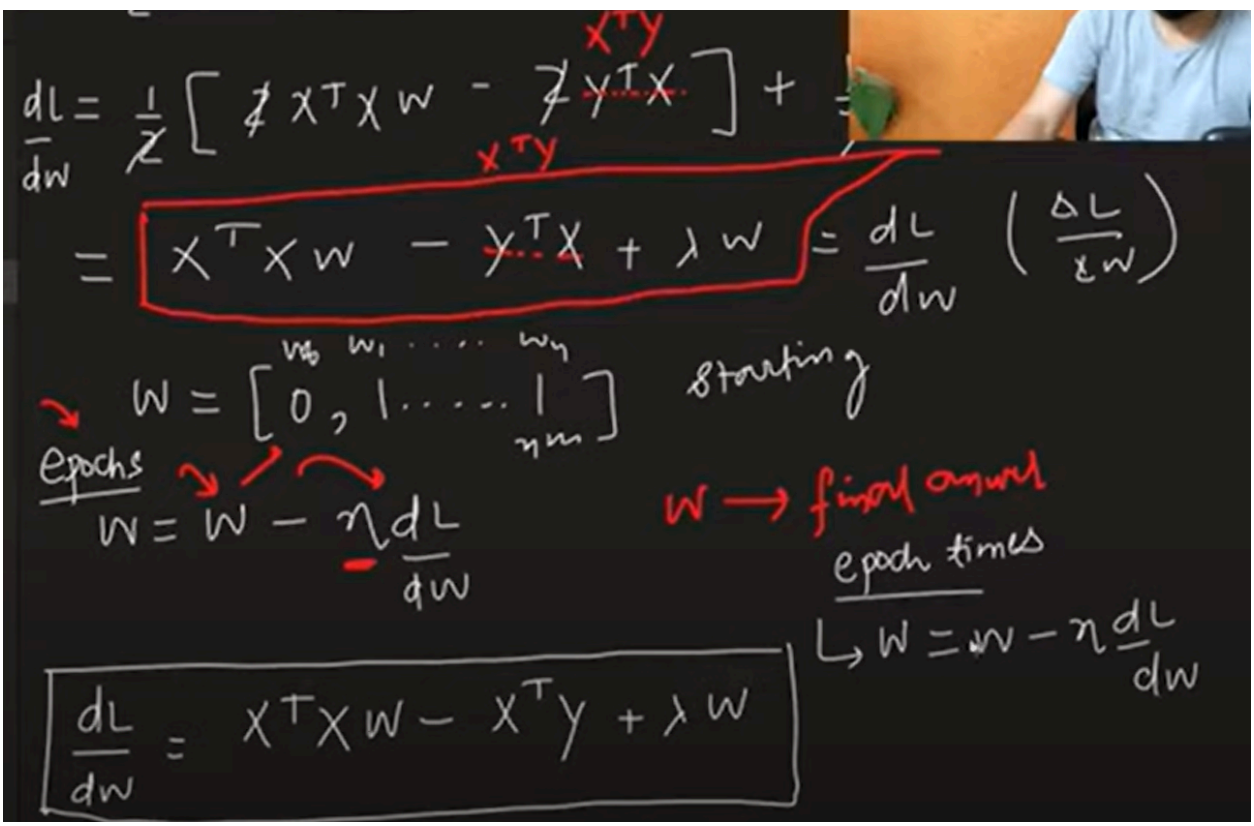

```
# Import required libraries
from sklearn.datasets import load_diabetes  # Diabetes datase
t
from sklearn.metrics import r2_score       # R-squared metric
import numpy as np                         # Numerical operat
ions

# Load diabetes dataset
X, y = load_diabetes(return_X_y=True)  # X contains features,
y contains target values

# Import train-test split function
from sklearn.model_selection import train_test_split

# Split data into training and test sets (80% train, 20% tes
t)
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2, random_state=4)
```

```
# Import Ridge regression from scikit-learn
from sklearn.linear_model import Ridge

# Create Ridge regression model with alpha=0.1 and cholesky s
olver
reg = Ridge(alpha=0.1, solver='cholesky')

# Fit the model to training data
reg.fit(X_train, y_train)

# Make predictions on test data and calculate R-squared score
y_pred = reg.predict(X_test)
r2_score(y_test, y_pred)

# Print model coefficients and intercept
print(reg.coef_)      # Coefficient values for each feature
print(reg.intercept_) # The intercept term

# Custom Ridge Regression Implementation
class MeraRidge:

    def __init__(self, alpha=0.1):
        # Initialize with regularization strength (alpha)
        self.alpha = alpha    # Regularization parameter
        self.coef_ = None     # Will store coefficients
        self.intercept_ = None # Will store intercept

    def fit(self, X_train, y_train):
        # Add column of 1s for intercept term (bias)
        X_train = np.insert(X_train, 0, 1, axis=1)

        # Create identity matrix (for regularization)
        I = np.identity(X_train.shape[1])
        # Don't regularize the intercept term (first element)
        I[0][0] = 0
```

```
        # Closed-form solution for Ridge Regression:
        # θ = (XᵀX + αI)⁻¹ Xᵀy
        result = np.linalg.inv(
            np.dot(X_train.T, X_train) + self.alpha * I
        ).dot(X_train.T).dot(y_train)

        # Extract intercept (first term) and coefficients (re
maining terms)
        self.intercept_ = result[0]
        self.coef_ = result[1:]

    def predict(self, X_test):
        # Make predictions using: ŷ = Xθ + intercept
        return np.dot(X_test, self.coef_) + self.intercept_

# Test custom implementation
reg = MeraRidge()
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)

# Verify results match scikit-learn's implementation
print(r2_score(y_test, y_pred))  # Should match previous R² s
core
print(reg.coef_)                 # Should match previous coef
ficients
print(reg.intercept_)            # Should match previous inte
rcept
```

For gradient descent ridge regression


$w_{new} = w_{old} - \eta \frac{\Delta L}{\Delta w}$ → gradient
L, w
$\partial L / \partial w_n$
$L = \frac{1}{2}(Xw - y)^T (Xw - y) + \frac{1}{2}\lambda w^T w$
$= \frac{1}{2}(w^T X^T - y^T)(Xw - y) + \frac{1}{2}\lambda w^T w$
$= \frac{1}{2}\left[w^T X^T X w - w^T X^T y - y^T w X + y^T y\right] + \frac{1}{2}\lambda w^T w$


$\frac{dL}{dw} = \frac{1}{2}\left[2X^T X w - 2X^T y\right] +$
$X^T y$
$= X^T X w - X^T y + \lambda w = \frac{dL}{dw}$ $\left(\frac{\Delta L}{\Delta w}\right)$
w_0 w_1 ..... w_n
$w = [0, 1 \ldots\ldots 1]$ starting
epochs
$w = w - \eta \frac{dL}{dw}$
$w \rightarrow$ final answer
epoch times
$\hookrightarrow w = w - \eta \frac{dL}{dw}$
$\frac{dL}{dw} = X^T X w - X^T y + \lambda w$

```python
# Import all necessary libraries at once
from sklearn.datasets import load_diabetes
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor, Ridge
import numpy as np

# Load and prepare the diabetes dataset
X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2, random_state=4)

# ================================================
# 1. Ridge Regression using SGDRegressor
# ================================================
print("\n=== SGDRegressor (L2 Penalty) ===")
sgd_reg = SGDRegressor(
    penalty='l2',           # L2 regularization (Ridge)
    max_iter=500,           # Number of epochs
    eta0=0.1,              # Initial learning rate
    learning_rate='constant',  # Fixed learning rate
    alpha=0.001            # Regularization strength
)
sgd_reg.fit(X_train, y_train)
y_pred = sgd_reg.predict(X_test)

print(f"R2 score: {r2_score(y_test, y_pred):.4f}")
print("Coefficients:", sgd_reg.coef_)
print("Intercept:", sgd_reg.intercept_)

# ================================================
# 2. Ridge Regression using scikit-learn's Ridge
# ================================================
print("\n=== Scikit-learn Ridge ===")
```

```
ridge_reg = Ridge(
    alpha=0.001,       # Regularization strength
    max_iter=500,      # Maximum iterations
    solver='sparse_cg'  # Solver for optimization
)
ridge_reg.fit(X_train, y_train)
y_pred = ridge_reg.predict(X_test)

print(f"R2 score: {r2_score(y_test, y_pred):.4f}")
print("Coefficients:", ridge_reg.coef_)
print("Intercept:", ridge_reg.intercept_)

# ================================================
# 3. Custom Ridge Regression Implementation
# ================================================
class MeraRidgeGD:
    """Custom Ridge Regression implementation using Gradient
Descent"""

    def __init__(self, epochs, learning_rate, alpha):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.alpha = alpha
        self.coef_ = None
        self.intercept_ = None

    def fit(self, X_train, y_train):
        # Initialize weights and intercept
        self.coef_ = np.ones(X_train.shape[1])
        self.intercept_ = 0
        thetha = np.insert(self.coef_, 0, self.intercept_)

        # Add bias term to X_train
        X_train = np.insert(X_train, 0, 1, axis=1)

        # Gradient Descent optimization
```

```
        for _ in range(self.epochs):
            # Calculate gradient with L2 regularization
            thetha_der = (np.dot(X_train.T, X_train).dot(thet
ha) -
                          np.dot(X_train.T, y_train) +
                          self.alpha * thetha)
            thetha -= self.learning_rate * thetha_der

        # Extract final coefficients and intercept
        self.coef_ = thetha[1:]
        self.intercept_ = thetha[0]

    def predict(self, X_test):
        return np.dot(X_test, self.coef_) + self.intercept_

print("\n=== Custom Ridge Implementation ===")
custom_ridge = MeraRidgeGD(
    epochs=500,
    alpha=0.001,
    learning_rate=0.005
)
custom_ridge.fit(X_train, y_train)
y_pred = custom_ridge.predict(X_test)

print(f"R2 score: {r2_score(y_test, y_pred):.4f}")
print("Coefficients:", custom_ridge.coef_)
print("Intercept:", custom_ridge.intercept_)
```

Important to watch
https://www.youtube.com/watch?v=8osKeShYVRQ&list=PLKnIA16_Rmvbr7zKYQuBfsVkjoLcJgxHH&index=65

```
# Importing necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Creating a synthetic dataset
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

# Plotting the dataset
plt.scatter(X, y, color="blue", alpha=0.7)
plt.xlabel("X")
plt.ylabel("y")
plt.title("Synthetic Dataset")
plt.show()

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2, random_state=42)

# Training a Ridge Regression model
alpha_values = [0.1, 1, 10]
models = {}

for alpha in alpha_values:
    ridge_reg = Ridge(alpha=alpha)
    ridge_reg.fit(X_train, y_train)
    models[alpha] = ridge_reg

# Calculating Mean Squared Error for each model
for alpha, model in models.items():
    y_pred = model.predict(X_test)
```

```
    mse = mean_squared_error(y_test, y_pred)
    print(f"Alpha: {alpha}, MSE: {mse}")

# Plotting regression lines
plt.scatter(X, y, color="blue", alpha=0.7, label="Data")
for alpha, model in models.items():
    plt.plot(X, model.predict(X), label=f"Alpha: {alpha}")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.title("Effect of Regularization")
plt.show()
```